

Introduction

Organizor is the backend API for managing tenant environments. It keeps tenant records in a central catalog database and provisions a separate PostgreSQL database for each tenant.

The current implementation focuses on tenant lifecycle operations:

- creating tenant catalog records;
- onboarding tenants by creating a tenant database and applying migrations through internal operations;
- resolving tenant context for tenant-scoped requests;
- upgrading tenant databases by running migrations;
- soft-deleting tenants before hard deletion;
- exposing health checks, OpenAPI JSON, and Scalar endpoint exploration in development.

Main Concepts

CONCEPT	DESCRIPTION
Tenant	A customer or isolated workspace tracked by <code>Id</code> , <code>Name</code> , <code>Slug</code> , <code>DatabaseName</code> , lifecycle status, and deletion metadata.
Catalog database	Central PostgreSQL database that stores all tenant records.
Tenant database	PostgreSQL database named from the tenant slug, currently using the <code>organizor_tenant_{slug}</code> convention.
Tenant resolution	Middleware step that reads <code>X-Tenant-Slug</code> , loads the tenant, and stores the current tenant in <code>ITenantContext</code> .
Provisioning workflow	Infrastructure workflow that creates the tenant database and applies tenant migrations.

Project Layout

PROJECT	RESPONSIBILITY
<code>Code7Dev.Organizor.Api</code>	ASP.NET Core host, controllers, DTOs, mappers, middleware, OpenAPI/Scalar, health checks, CORS, and logging setup.
<code>Code7Dev.Organizor.Application</code>	Tenant service contracts, tenant models, onboarding, lifecycle, upgrade, and catalog-facing application logic.
<code>Code7Dev.Organizor.Domain</code>	Tenant entity and <code>TenantStatus</code> enum.
<code>Code7Dev.Organizor.Infrastructure</code>	EF Core contexts, repository implementation, PostgreSQL connection string building, database creation/deletion, migration, and tenant context storage.

Documentation Boundaries

The documentation is split into two intentionally different areas:

AREA	PURPOSE
Conceptual docs	Explain behavior, workflows, configuration, security, local development, operations, and integration decisions.
Generated API reference	Lists C# namespaces, classes, interfaces, methods, and signatures generated from source.

Do not use the generated reference to explain workflows or product behavior. Keep that content in the conceptual articles so it stays readable and can include context, examples, and operational guidance.



Getting Started

For the full local topology, expected ports, startup order, and verification checklist, use [Local Development](#). This page keeps the shorter first-run flow.

Prerequisites

- .NET SDK 10.
- PostgreSQL available to the API.
- Keycloak available to the API. The repository includes a development Keycloak compose file.
- A catalog database connection string named `Catalog`.
- PostgreSQL server settings under the `Postgres` configuration section for tenant database creation and resolution.
- Keycloak bearer-token settings under the `Keycloak` configuration section.
- DocFX installed as a .NET tool when building documentation locally.

Install DocFX if the `docfx` command is not available:

```
dotnet tool install -g docfx
```

Start the development Keycloak instance:

```
docker compose up -d
```

The compose file starts Keycloak at `https://localhost:8443`. Generate and trust the local certificate first if `keycloak/certs/keycloak.p12` does not exist. Configure the `organizer-dev` realm, clients, roles, and users as described in [Keycloak Setup](#).

Run the API

From the repository root:

```
dotnet run --project .\Code7Dev.Organizor.Api\Code7Dev.Organizor.Api.csproj
```

Development launch profiles expose:

PROFILE	URL
http	http://localhost:5100
https	https://localhost:7184 and http://localhost:5100

On startup the API:

- 1. configures Serilog;
- 2. registers catalog and tenant database services;
- 3. registers application and infrastructure services;
- 4. seeds or migrates the catalog database;
- 5. maps controllers, health checks, OpenAPI, and Scalar in development.

Development Endpoints

ENDPOINT	PURPOSE
/health	ASP.NET Core health checks.
/openapi/v1.json	OpenAPI document, available in development.
/scalar	Interactive OpenAPI/Scalar endpoint explorer, available in development.

Example tenant catalog request:

```
$token = "<organizer-admin JWT>"
```

```
Invoke-RestMethod `
-Method Post `
-Uri "http://localhost:5100/api/v1/tenants" `
-Headers @{ Authorization = "Bearer $token" } `
-ContentType "application/json" `
-Body '{"name":"Acme","slug":"acme"}'
```

`POST /api/v1/tenants` creates a catalog record only and requires an `organizer-admin` token. Use `POST /api/admin/tenants/onboard` when you also need tenant database and Keycloak provisioning.

Run the Portal UI

From the repository root:

```
dotnet run --project
.\Code7Dev.Organizer.Portal.Ui\Code7Dev.Organizer.Portal.Ui.csproj --
launch-profile https
```

The HTTPS profile exposes the portal at `https://localhost:7025`. The portal is configured to:

- authenticate against `https://localhost:8443/realms/organizer-dev`;
- use the `organizer-ui` Keycloak client;
- receive login callbacks at `https://localhost:7025/authentication/login-callback`;
- call the API at `https://localhost:7184`.

Build the Documentation

From the repository root:

```
docfx .\docs\docfx.json
```

DocFX writes the generated site to `docs/_site` and generated C# type-reference metadata to `docs/api`. Both folders are ignored by git and can be rebuilt.

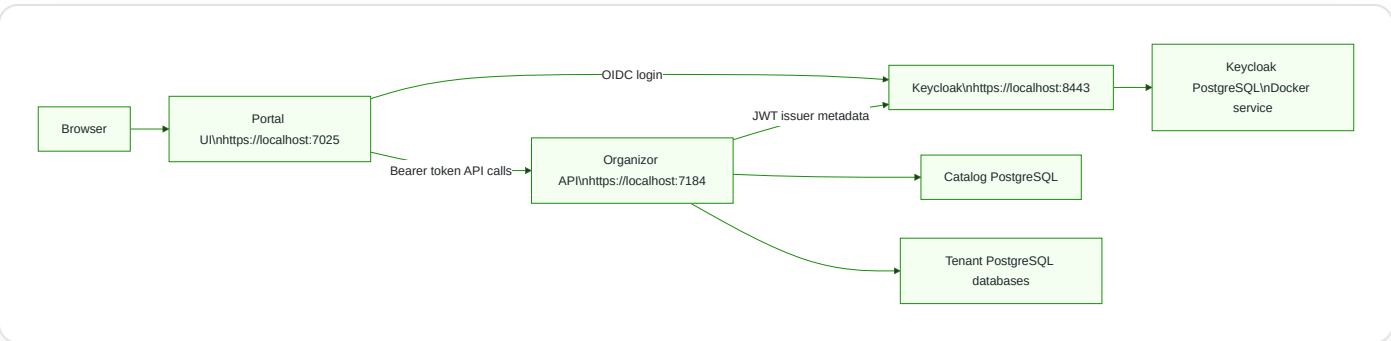
Local Development

This page is the canonical reference for running Organizer locally. Use it to confirm the expected ports, startup order, and how the Blazor portal, Keycloak, API, and databases fit together.

NOTE

Keep this page as the source of truth for local ports and startup order. Link here from other articles instead of duplicating endpoint tables.

Local Topology



Ports And URLs

COMPONENT	URL	NOTES
Portal UI	https://localhost:7025	Blazor WebAssembly HTTPS launch profile
Portal UI HTTP	http://localhost:5080	Development fallback profile. Prefer HTTP! for auth testing.

COMPONENT	URL	NOTES
Organizor API	<code>https://localhost:7184</code>	API HTTPS launch profile used by the portal.
Organizor API HTTP	<code>http://localhost:5100</code>	Useful for direct local API calls without the portal.
Keycloak	<code>https://localhost:8443</code>	Docker Compose service with local HTTPS certificate.
Keycloak realm issuer	<code>https://localhost:8443/realms/organizor-dev</code>	Must match API <code>Keycloak:Authority</code>
Keycloak token endpoint	<code>https://localhost:8443/realms/organizor-dev/protocol/openid-connect/token</code>	Used for manual token testing.

Required Local Configuration

The API needs:

- a `ConnectionStrings:Catalog` connection string;
- `Postgres` settings for tenant database creation and resolution;
- `Keycloak:Authority` set to `https://localhost:8443/realms/organizor-dev`;
- `Keycloak:BaseUrl` set to `https://localhost:8443`;
- `Keycloak:Realm` set to `organizor-dev`;
- `Keycloak:AdminClientId` set to `organizor-api`;
- `Keycloak:AdminClientSecret` from the Keycloak `organizor-api` client.

⚠ WARNING

The API authority must match the token issuer exactly. A token issued by `https://localhost:8443/realms/organizor-dev` will not validate against any

different scheme, host, port, or realm.

The portal is currently configured to:

- use Keycloak authority `https://localhost:8443/realms/organizor-dev`;
- use Keycloak client ID `organizor-ui`;
- redirect login callbacks to `https://localhost:7025/authentication/login-callback`;
- call the API at `https://localhost:7184`.

Startup Order

1. Generate or verify the Keycloak HTTPS certificate at `keycloak/certs/keycloak.p12`.
2. Start Keycloak with Docker Compose.
3. Confirm the `organizor-dev` realm, clients, roles, groups, and users exist.
4. Start the API with the HTTPS launch profile.
5. Start the portal with the HTTPS launch profile.
6. Sign in through the portal and verify the API accepts the issued token.

Commands

Start Keycloak:

```
docker compose up -d
```

Run the API:

```
dotnet run --project .\Code7Dev.Organizor.Api\Code7Dev.Organizor.Api.csproj  
--launch-profile https
```

Run the portal:

```
dotnet run --project
.\Code7Dev.Organizor.Portal.Ui\Code7Dev.Organizor.Portal.Ui.csproj --
launch-profile https
```

Build the documentation:

```
docfx .\docs\docfx.json
```

Keycloak Client Requirements

The `organizor-ui` client must allow the portal origin:

SETTING	VALUE
Valid redirect URI	<code>https://localhost:7025/*</code>
Login callback	<code>https://localhost:7025/authentication/login-callback</code>
Web origin	<code>https://localhost:7025</code>

The `organizor-api` client is used for API service-account operations during tenant onboarding and hard deletion. Its client secret must be available to the API through configuration or user secrets.

PRODUCTION CHECKLIST

Do not copy local wildcard origins, local certificates, or development secrets into deployed environments. Production should use real HTTPS certificates, explicit origins, and managed secret storage.

Verification Checklist

Use this checklist when local authentication or API calls fail:

- Keycloak opens at `https://localhost:8443` without a certificate warning.
- The API `Keycloak:Authority` exactly matches `https://localhost:8443/realms/organizor-dev`.
- The portal login callback is allowed by the `organizor-ui` client.
- The portal calls `https://localhost:7184`, not the API HTTP endpoint.
- The API HTTPS profile is running before logging in through the portal.
- The signed-in user has the expected realm roles and tenant groups.
- The portal user information panel shows the expected `tenant`, `preferred_username`, `email`, and `groups` claim values.

Related Guides

- [Keycloak HTTPS Setup](#) explains certificate generation and browser trust.
- [Keycloak Setup](#) explains realm, client, role, group, and claim mapper configuration.
- [Configuration](#) documents API configuration sections.
- [Portal UI Overview](#) documents the Blazor portal responsibilities and integration points.

Portal UI Overview

The Portal UI is the Blazor WebAssembly front end for Organizer. It handles Keycloak sign-in, role-based routing, tenant administration, and authenticated API calls to the backend.

This page reflects the issue 98 portal work completed after the last documentation update.

Current Capabilities

The portal currently supports:

- Keycloak OIDC login with the `organizer-ui` client.
- Role extraction from Keycloak realm roles and `organizer-ui` client roles.
- Automatic role-based routing after login.
- Separate admin and tenant shells.
- An admin dashboard route at `/admin`.
- Tenant administration at `/admin/tenants` for platform administrators.
- Tenant plan default management at `/admin/plans`.
- Tenant effective config inspection and override editing from `/admin/tenants`.
- Read-only tenant effective config at `/tenant/config`.
- Logout through `/logout` or the shared logout button.
- Current-user display with username, email, tenant claim, and Keycloak groups.
- Tenant dashboard/profile placeholders under `/tenant`.

The `/admin/tenants`, `/admin/plans`, and `/tenant/config` pages are backed by real API routes. They are not mock UI.

Technology Stack

- **Blazor WebAssembly** for client-side rendering.
- **.NET 10** for the portal project.
- **Keycloak** for authentication using OIDC Authorization Code Flow with PKCE.

- **Radzen.Blazor** for the admin tenant grid, cards, forms, buttons, badges, alerts, and notifications.
- **IHttpClientFactory** plus a custom `AuthorizationMessageHandler` for API calls.

Routes and Layouts

ROUTE	LAYOUT	REQUIRED ROLE	PURPOSE
<code>/</code>	<code>MainLayout</code>	none	Entry page. Authenticated users are redirected by role.
<code>/admin</code>	<code>AdminLayout</code>	<code>organizer-admin</code>	Admin dashboard placeholder.
<code>/admin/tenants</code>	<code>AdminLayout</code>	<code>organizer-admin</code>	Tenant catalog and lifecycle management.
<code>/admin/plans</code>	<code>AdminLayout</code>	<code>organizer-admin</code>	Tenant plan default management.
<code>/tenant</code>	<code>TenantLayout</code>	<code>tenant-admin</code>	Tenant dashboard placeholder.
<code>/tenant/profile</code>	<code>TenantLayout</code>	<code>tenant-admin</code>	Tenant profile placeholder.
<code>/tenant/config</code>	<code>TenantLayout</code>	<code>tenant-admin</code> , <code>tenant-user</code>	Read-only effective tenant configuration.
<code>/logout</code>	none	authenticated user	Redirects to the Blazor OIDC logout

ROUTE	LAYOUT	REQUIRED ROLE	PURPOSE
<code>/authentication/*</code>	authentication page	none	endpoint and returns to <code>/</code>. Blazor WebAssembly OIDC endpoints.

The admin shell currently contains a navigation link for `/admin/users`. That is a navigation placeholder only; this branch does not add a backing Razor page for that route.

`RoleRedirector` is mounted globally in `App.razor`, but it only redirects from the neutral entry routes (`/` and the login callback). It sends `organizer-admin` users to `/admin`, `tenant-admin` users to `/tenant`, and `tenant-user` users to `/tenant/config`. It must not redirect when the user is already on a target area, otherwise the portal can reload-loop.

Admin Tenant Management

The admin tenant page uses Radzen components and the `AdminTenantService` client. It supports:

- loading all tenants including soft-deleted tenants;
- filtering tenants by name, slug, database name, or status;
- viewing active, deleted, and failed tenant counts;
- onboarding a tenant with name and slug validation;
- upgrading a tenant database by running pending migrations;
- soft-deleting a tenant with an operator reason;
- hard-deleting a previously soft-deleted tenant.
- inspecting effective tenant config;
- editing tenant plan assignment and tenant-level overrides.

The page calls these backend endpoints:

UI ACTION	API ENDPOINT
Load tenants	GET /api/admin/tenants
Load one tenant	GET /api/admin/tenants/{id}
Onboard tenant	POST /api/admin/tenants/onboard
Upgrade tenant	POST /api/admin/tenants/{id}/upgrade
Inspect effective config	GET /api/admin/tenants/{id}/config
Load tenant overrides	GET /api/admin/tenants/{id}/config-overrides
Save tenant overrides	PUT /api/admin/tenants/{id}/config-overrides
Soft delete tenant	DELETE /api/admin/tenants/{id}
Hard delete tenant	POST /api/admin/tenants/{id}/hard-delete

All admin tenant operations require a valid bearer token with the `organizer-admin` role.

Admin Plan Management

The admin plans page uses `AdminTenantService` and supports:

- loading tenant plan definitions;
- editing plan name, description, active state, sort order, feature defaults, and settings defaults;
- saving plan changes through the admin plan API.

The page calls these backend endpoints:

UI ACTION	API ENDPOINT
Load plans	GET /api/admin/tenant-plans
Save plan defaults	PUT /api/admin/tenant-plans/{planKey}

Plan keys are intentionally immutable.

Tenant Config Page

Tenant admins and tenant users can open /tenant/config . The page calls GET /api/v1/tenant/config through TenantConfigService .

ApiAuthorizationMessageHandler adds X-Tenant-Slug from the authenticated user's tenant claim.

Tenant rows expose the catalog and lifecycle fields used by the UI:

FIELD	USE
id	Action target for upgrade and delete calls.
name , slug , databaseName	Catalog display and search.
createdAt	Created timestamp shown in local time.
status	Lifecycle state used for badges and failed counts.
failureReason	Failure detail shown below the tenant name when present.
isDeleted	Controls deleted badge state and whether hard delete is available.

Authentication Flow

The Portal UI uses:

- OIDC Authorization Code Flow with PKCE;
- Keycloak as the identity provider;
- access tokens for API calls;
- Blazor `AuthenticationState` for role and user context;
- `KeycloakClaimsPrincipalFactory` to promote Keycloak roles into Blazor role claims.

Current local configuration:

SETTING	VALUE
Portal origin	<code>https://localhost:7025</code>
Keycloak authority	<code>https://localhost:8443/realms/organizer-dev</code>
Keycloak client	<code>organizer-ui</code>
Login callback	<code>https://localhost:7025/authentication/login-callback</code>
Logout callback	<code>https://localhost:7025/</code>
API base URL	<code>https://localhost:7184</code>

The API `Keycloak:Authority` must match the token issuer exactly. A portal token issued by `https://localhost:8443/realms/organizer-dev` will fail API validation if the API is configured for `http://localhost:8443/realms/organizer-dev`.

API Integration

The portal uses a named `HttpClient` called `api` with base address `https://localhost:7184`.

`ApiAuthorizationMessageHandler` :

- configures the authorized API URL;
- requests the OIDC scopes used by the portal;
- attaches bearer tokens to API requests;
- adds `X-Tenant-Slug` when `UserContextService` can resolve a tenant claim.

The handler is attached with

`.AddHttpMessageHandler<ApiAuthorizationMessageHandler>()` and configured once in its constructor. Do not configure the same `AuthorizationMessageHandler` instance more than once; Blazor will throw a handler already configured error.

Summary

The portal now supports tenant lifecycle operations, tenant plan defaults, tenant-level config overrides, and read-only tenant config inspection. User management remains a placeholder until backing API use cases are added.

Authentication Flow — Portal UI

The Portal UI uses Keycloak to authenticate users and determine which shell they should see. The process follows OpenID Connect Authorization Code Flow with PKCE.

Login Flow

1. The user clicks **Login** or navigates to a protected route.
2. The Portal redirects to Keycloak at `https://localhost:8443/realms/organizor-dev`.
3. Keycloak validates the credentials.
4. Keycloak redirects back to `https://localhost:7025/authentication/login-callback`.
5. Blazor WebAssembly stores the authenticated state and access token.
6. `KeycloakClaimsPrincipalFactory` maps Keycloak roles into Blazor role claims.
7. `RoleRedirector` sends the user to the correct shell when they return from the neutral entry routes.

Role Redirects

`RoleRedirector` currently routes users as follows:

ROLE	REDIRECT TARGET
<code>organizor-admin</code>	<code>/admin</code>
<code>tenant-admin</code>	<code>/tenant</code>
<code>tenant-user</code>	<code>/tenant/config</code>

The redirector must only redirect from neutral routes such as `/` and `authentication/login-callback`. It should not force navigation when the user is

already on `/admin` or `/tenant`, because forced self-navigation can create a reload loop.

Logout is initiated either by `LogoutButton` or the `/logout` route. Both call Blazor WebAssembly authentication through `NavigateToLogout("authentication/logout", "/")`, which clears the local authentication state and returns the browser to `/` after the identity-provider logout flow.

Role Claims in the Portal

The portal role factory reads roles from Keycloak claims and adds `ClaimTypes.Role` entries for Blazor authorization.

Supported sources:

- `realm_access.roles`
- `resource_access.organizor-ui.roles`

This is separate from API role mapping, where the API also understands roles under `resource_access.organizor-api.roles`.

Protected Routes

Portal pages use role attributes:

```
@attribute [Authorize(Roles = "organizor-admin")]
```

Current route protection:

ROUTE AREA	REQUIRED ROLE
<code>/admin</code>	<code>organizor-admin</code>
<code>/admin/tenants</code>	<code>organizor-admin</code>
<code>/admin/plans</code>	<code>organizor-admin</code>

ROUTE AREA	REQUIRED ROLE
/tenant	tenant-admin
/tenant/profile	tenant-admin
/tenant/config	tenant-admin , tenant-user

The admin layout still contains a placeholder navigation link for `/admin/users`. There is no Razor page for that route yet.

API Tokens

The Portal never sends credentials to the API. It sends bearer tokens through the configured API `HttpClient`. The API validates token signature, issuer, expiration, and then applies authorization policies.

The local issuer must be consistent everywhere:

```
https://localhost:8443/realms/organizer-dev
```

If the API is configured with `http://localhost:8443/realms/organizer-dev`, it will reject HTTPS-issued portal tokens with `401 Unauthorized`.

Keycloak Client Requirements

The `organizer-ui` client must allow:

SETTING	VALUE
Valid redirect URI	<code>https://localhost:7025/*</code>
Web origin	<code>https://localhost:7025</code>

SETTING	VALUE
Flow	Standard flow / Authorization Code with PKCE

Users must receive the appropriate Organizer role before logging in. If roles are changed in Keycloak, log out and back in to refresh the token and claims.

The current-user display reads these claims when present:

CLAIM	USE
preferred_username	Display name fallback before email.
email	User contact display.
tenant	Tenant context and X-Tenant-Slug header source.
groups	Sidebar user context display.

Portal UI Structure — High-Level

The Portal UI is organized into pages, layouts, components, services, and models. Issue 98 added the first operational admin workflow for tenant control.

Pages

PAGE	ROUTE	RESPONSIBILITY
Home	/	Neutral entry page. Authenticated users can be redirected by role.
Authentication	/authentication/*	Blazor WebAssembly OIDC login/logout callbacks.
Logout	/logout	Convenience route that starts Blazor OIDC logout and returns to <code>/</code> .
Admin/Dashboard	/admin	Admin dashboard placeholder.
Admin/Tenants	/admin/tenants	Operational tenant catalog and lifecycle management.
Admin/Plans	/admin/plans	Tenant plan default management.
Tenant/Dashboard	/tenant	Tenant dashboard placeholder.
Tenant/Profile	/tenant/profile	Tenant profile placeholder.
Tenant/Config	/tenant/config	Read-only effective tenant configuration.
NotFound	fallback	Not-found page.

`AdminLayout` includes a navigation link for `/admin/users`, but there is no Razor page for that route yet. Treat it as a future navigation placeholder.

Layouts

LAYOUT	RESPONSIBILITY
<code>MainLayout</code>	General shell with login/logout controls and authenticated user context.
<code>AdminLayout</code>	Admin control-plane shell with dashboard, tenant management, current-user panel, logout, and public-site navigation.
<code>TenantLayout</code>	Tenant shell with tenant navigation, current-user panel, logout, and public-site navigation.

Components

COMPONENT	RESPONSIBILITY
<code>UserInfo</code>	Displays tenant, username, email, and Keycloak groups from the current authentication state.
<code>CurrentUserPanel</code>	Sidebar user card used in admin and tenant shells. Shows display name, email, tenant, groups, logout, and back-to-website action.
<code>LogoutButton</code>	Starts Blazor OIDC logout through <code>NavigateToLogout("authentication/logout", "/")</code> .
<code>RoleRedirector</code>	Redirects authenticated users from neutral entry routes to <code>/admin</code> , <code>/tenant</code> , or <code>/tenant/config</code> based on role.

COMPONENT	RESPONSIBILITY
<code>RedirectToLogin</code>	Sends unauthenticated users to the Blazor OIDC login route.
<code>AuthStateProvider</code>	Auth-related component wrapper used by the portal.
<code>AdminSidebar</code>	Admin navigation component retained for admin navigation reuse.

Radzen Usage

The portal references `Radzen.Blazor` and wires `RadzenComponents` in `App.razor`.

The admin tenant page currently uses:

- `RadzenDataGrid` for tenant catalog display;
- `RadzenTemplateForm`, `RadzenTextBox`, `RadzenTextArea`, and validators for tenant onboarding and deletion input;
- `RadzenCard` for layout panels;
- `RadzenButton` for actions;
- `RadzenBadge` for tenant status;
- `RadzenAlert` and `NotificationService` for errors and operation feedback.

Radzen services registered in `Program.cs`:

- `DialogService`
- `NotificationService`
- `TooltipService`
- `ContextMenuService`

Services

SERVICE	RESPONSIBILITY
UserContextService	Reads <code>AuthenticationState</code> and maps claims into <code>UserInfoModel</code> .
ApiAuthorizationMessageHandler	Attaches access tokens to API requests and adds tenant header context when available.
AdminTenantService	Typed client for admin tenant, plan, config inspection, and override management operations.
TenantConfigService	Typed client for the tenant-scoped effective config endpoint used by <code>/tenant/config</code> .
KeycloakClaimsPrincipalFactory	Promotes Keycloak roles into Blazor role claims.

Models

MODEL	PURPOSE
<code>UserInfoModel</code>	Display model for signed-in user context.
<code>TenantModel</code>	Portal-side tenant catalog projection.
<code>TenantOnboardingForm</code>	Validated form model for tenant onboarding.
<code>TenantOnboardingRequest</code>	API request body for onboarding.
<code>TenantSoftDeleteRequest</code>	API request body for soft deletion.
<code>TenantMigrationResult</code>	API response model for tenant database upgrades.

MODEL

TenantEffectiveConfigModel

TenantPlanModel

TenantConfigOverridesModel

PURPOSE

Effective tenant config response model used by tenant and admin config views.

Tenant plan defaults model used by `/admin/plans`.

Tenant plan assignment and override model used by `/admin/tenants`.

TenantModel mirrors the API tenant response used by the admin grid: `Id`, `Name`, `Slug`, `DatabaseName`, `CreatedAt`, `Status`, `FailureReason`, and `IsDeleted`.

Extension Rules

- Add page-level authorization attributes to protected pages.
- Keep API calls inside services instead of Razor markup.
- Use the named `api` HttpClient for backend calls so tokens are attached consistently.
- Avoid forcing full page reloads for internal route redirects unless a complete browser reload is required.
- Do not add UI for operations until the backing API route exists, unless the page is clearly marked as a placeholder.

API Integration — Portal UI

The Portal UI communicates with the Organizer API through a named `HttpClient` called `api`. All tenant administration actions go through this client.

Local Development Configuration

SETTING	VALUE
Portal origin	<code>https://localhost:7025</code>
API base URL	<code>https://localhost:7184</code>
Authorized API URL	<code>https://localhost:7184</code>
Token issuer	<code>https://localhost:8443/realms/organizer-dev</code>
Portal OIDC client	<code>organizer-ui</code>

The portal and API must agree on the Keycloak issuer. If the portal logs in through HTTPS but the API validates against the old HTTP authority, API requests fail with `401 Unauthorized`.

HttpClient Registration

The portal registers the API client in `Program.cs`:

```
builder.Services.AddHttpClient("api", client =>
{
    client.BaseAddress = new Uri("https://localhost:7184");
})
.AddHttpMessageHandler<ApiAuthorizationMessageHandler>();
```

`AdminTenantService` and `TenantConfigService` use `IHttpClientFactory.CreateClient("api")` to call API endpoints.

Authorization Handler

`ApiAuthorizationMessageHandler` extends Blazor's `AuthorizationMessageHandler`. It is responsible for:

- configuring the authorized API URL;
- requesting the portal OIDC scopes;
- attaching the access token to outgoing API requests;
- adding the `X-Tenant-Slug` header when a tenant claim is available.

The handler is registered in DI and configured once in its constructor. This matters because `AuthorizationMessageHandler.ConfigureHandler(...)` can only be called once per handler instance. Calling `ConfigureHandler(...)` again for the same handler instance can cause this runtime error:

```
handler already configured
```

Keep handler configuration inside the handler constructor or inside the `AddHttpMessageHandler` factory, but not both. The current branch uses the constructor approach.

Tenant Admin API Client

`AdminTenantService` wraps the admin tenant API surface used by `/admin/tenants` and `/admin/plans`. Tenant query, lifecycle, effective config, override, and plan-management endpoints require the Organizer admin policy.

METHOD	ENDPOINT	PURPOSE
<code>GetTenantsAsync</code>	<code>GET /api/admin/tenants</code>	Loads all tenants,

METHOD	ENDPOINT	PURPOSE
OnboardAsync	POST /api/admin/tenants/onboard	including soft-deleted records. Creates the catalog tenant, provisions Keycloak resources, creates the database, and applies migrations.
UpgradeAsync	POST /api/admin/tenants/{id}/upgrade	Runs pending migrations for the tenant database.
SoftDeleteAsync	DELETE /api/admin/tenants/{id}	Marks a tenant as deleted without dropping the database.
HardDeleteAsync	POST /api/admin/tenants/{id}/hard-delete	Drops the tenant database and removes the catalog

METHOD	ENDPOINT	PURPOSE
GetConfigAsync	GET /api/admin/tenants/{id}/config	record after soft deletion. Loads the tenant's effective config after plan defaults and overrides are resolved.
GetConfigOverridesAsync	GET /api/admin/tenants/{id}/config-overrides	Loads explicit tenant-level config overrides.
UpdateConfigOverridesAsync	PUT /api/admin/tenants/{id}/config-overrides	Updates tenant-level config overrides.
GetPlansAsync	GET /api/admin/tenant-plans	Loads tenant plan defaults.
UpdatePlanAsync	PUT /api/admin/tenant-plans/{planKey}	Updates tenant plan defaults.

The backend also exposes `GET /api/admin/tenants/{id}` for admin detail lookups. The current portal grid does not call that endpoint directly.

All methods expect an authenticated `organizer-admin` user. `401 Unauthorized` means the token is missing or invalid. `403 Forbidden` means the token is valid but lacks the required role.

Tenant query responses include lifecycle fields that the portal relies on:

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "name": "Acme",
  "slug": "acme",
  "databaseName": "organizer_tenant_acme",
  "createdAt": "2026-06-09T10:30:00Z",
  "status": "Active",
  "failureReason": null,
  "isDeleted": false
}
```

`GET /api/v1/tenants` returns only non-deleted catalog records. `GET /api/admin/tenants` intentionally includes soft-deleted records so the portal can show deleted tenants and allow hard deletion.

Tenant Config API Client

`TenantConfigService` wraps the tenant-scoped config endpoint used by `/tenant/config`.

METHOD	ENDPOINT	PURPOSE
<code>GetConfigAsync</code>	<code>GET /api/v1/tenant/config</code>	Loads the current tenant's effective config.

The request must include a bearer token with a tenant role and the `X-Tenant-Slug` header. `ApiAuthorizationMessageHandler` adds the tenant header when the

authenticated user has a tenant claim.

Error Handling

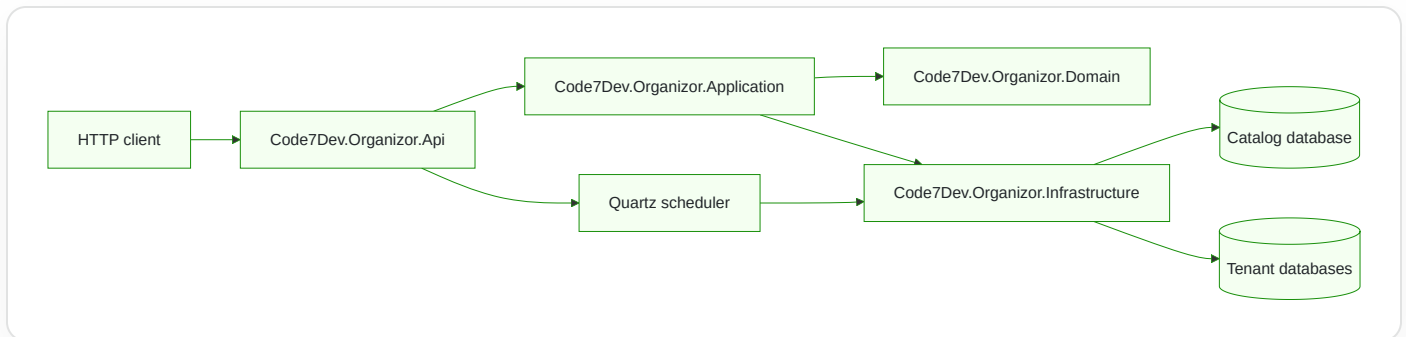
The tenant API client uses `EnsureSuccessAsync` to surface failed API responses as `InvalidOperationException`. The exception message includes the status code and response body when present. The Radzen tenant page catches those exceptions and displays both a page alert and a Radzen notification.

Troubleshooting

SYMPTOM	LIKELY CAUSE	FIX
<code>handler</code> already configured	The same <code>AuthorizationMessageHandler</code> instance was configured more than once.	Configure the handler in one place. The current branch configures it in <code>ApiAuthorizationMessageHandler</code> constructor.
<code>401</code> Unauthorized while logged in	API <code>Keycloak:Authority</code> does not match the token issuer, or no access token was attached.	Align API authority with <code>https://localhost:8443/realms/ dev</code> and verify the named API client in the auth handler.
<code>403</code> Forbidden while logged in	The token is valid but does not include <code>organizer-admin</code> .	Assign the role in Keycloak and to get a fresh token.
Tenant- scoped API returns bad tenant errors	Tenant header is missing or invalid.	Verify the user has a tenant claim. The <code>ApiAuthorizationMessageHandler</code> header for tenant-scoped endpoints.

Architecture

Organizor follows a layered structure around tenant management.



API Layer

The API project owns transport concerns:

- controller routes for public tenant catalog operations and internal tenant operations;
- request and response DTOs;
- mapping between API DTOs and application models;
- correlation ID, exception handling, and tenant resolution middleware;
- Serilog request logging;
- CORS, health checks, OpenAPI, and Scalar setup.

The startup flow in `Program.cs` composes the application by registering catalog persistence, tenant persistence, application services, infrastructure services, and API services.

Application Layer

The application project owns use cases and contracts. It defines interfaces for repository access, tenant database management, migration, provisioning, onboarding, upgrades, lifecycle operations, tenant config resolution, config overrides, and plan management.

Current services:

SERVICE	RESPONSIBILITY
TenantService	Creates catalog tenant records and reads tenants by slug or as a list.
TenantOnboardingService	Creates a catalog tenant, runs provisioning, and marks the tenant active or failed.
TenantLifecycleService	Soft-deletes tenants and hard-deletes already soft-deleted tenants.
TenantUpgradeService	Runs migrations for an existing tenant database.
TenantConfigResolver	Resolves effective tenant config from plan defaults plus tenant overrides.
TenantConfigManagementService	Reads and updates tenant-specific config overrides.
TenantPlanManagementService	Reads and updates tenant plan defaults.

Domain Layer

The domain project contains the tenant entity, tenant plan entity, tenant feature-settings value object, and tenant status enum. A tenant stores identity, slug, database name, plan key, config overrides, creation date, lifecycle status, provisioning failure reason, and deletion metadata. Tenant plans store reusable defaults for feature limits and behavior.

Infrastructure Layer

The infrastructure project owns external resources:

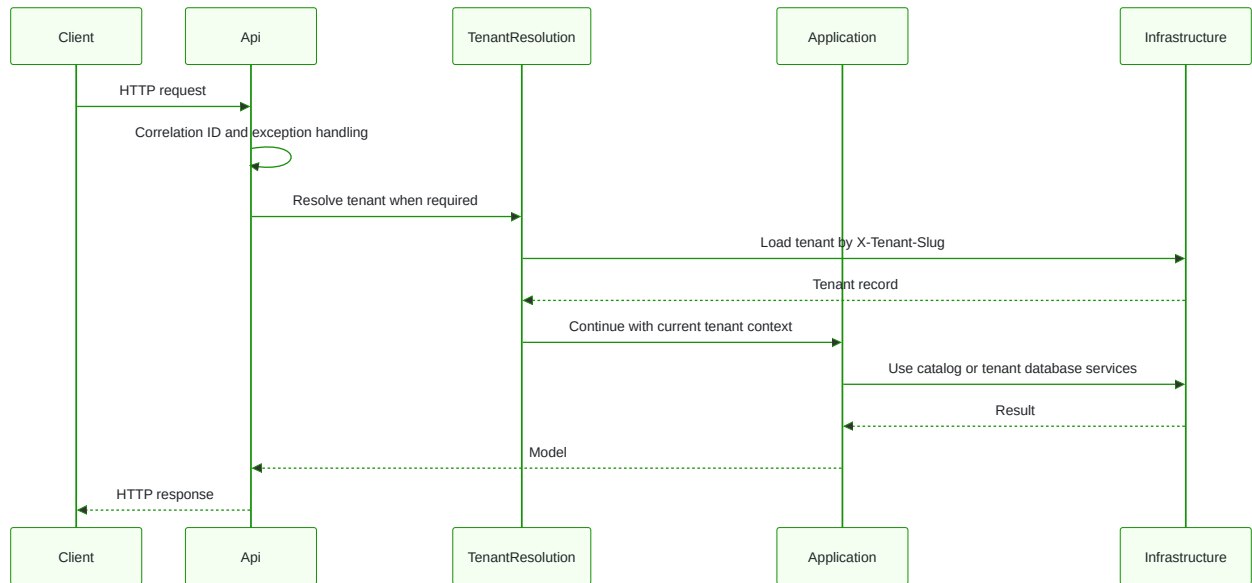
- `CatalogDbContext` stores tenant records.

- `OrganizorDbContext` connects to the current tenant database after tenant resolution.
- `TenantRepository` implements catalog tenant persistence.
- `TenantPlanRepository` implements tenant plan persistence.
- `ConnectionStringBuilder` builds PostgreSQL connection strings from configuration.
- `TenantDatabaseManager` creates and drops tenant databases.
- `TenantMigrator` applies EF Core migrations to tenant databases.
- `TenantProvisioningWorkflow` creates a tenant database and applies migrations during onboarding.
- `TenantJobScheduler` schedules tenant-scoped Quartz jobs from catalog tenant records.
- `TenantAwareJobFactory` creates Quartz jobs through dependency injection.
- `SyncTenantJob` runs recurring tenant synchronization work with `ITenantContext` populated from Quartz job data.

Background Jobs

Quartz is registered by the API layer and backed by persistent PostgreSQL storage in the catalog database. On startup, the application schedules a recurring synchronization job for each catalog tenant. Each job carries the tenant ID and slug in Quartz job data, then sets the tenant context before running tenant database work.

Request Pipeline



Tenancy

Organizor uses catalog-driven, database-per-tenant tenancy.

Tenant Catalog

The catalog database stores tenant metadata through `CatalogDbContext`. Each tenant includes:

- `Id` ;
- `Name` ;
- `Slug` ;
- `DatabaseName` ;
- `PlanKey` ;
- tenant feature overrides;
- tenant settings overrides;
- `CreatedAt` ;
- `Status` ;
- `FailureReason` ;
- soft-delete metadata.

The database name convention is `organizor_tenant_{slug}`. Slugs are normalized to lowercase before they are stored or used for lookups.

Normal catalog reads exclude soft-deleted tenants. Repository methods can include soft-deleted records when an internal operation explicitly asks for them.

Tenant Plans And Config Overrides

Tenant plans are persisted in the catalog database through the `tenant_plans` table. A plan defines default feature flags and structured settings for tenants assigned to that plan.

Each tenant stores:

FIELD	PURPOSE
PlanKey	Stable key of the selected plan, such as <code>starter</code> or <code>pro</code> .
Features	Tenant-level feature flag overrides stored as PostgreSQL <code>jsonb</code> .
Settings	Tenant-level structured setting overrides stored as PostgreSQL <code>jsonb</code> .

Each plan stores:

FIELD	PURPOSE
Key	Stable plan identifier referenced by tenants.
Name	Human-readable plan name.
Description	Optional operator-facing description.
Features	Plan default feature flags stored as PostgreSQL <code>jsonb</code> .
Settings	Plan default structured settings stored as PostgreSQL <code>jsonb</code> .
IsActive	Whether the plan can be assigned.
SortOrder	Display ordering for future admin UI surfaces.
CreatedAt	UTC creation timestamp.

The catalog migration creates a minimal `starter` plan before adding the tenant foreign key. This allows existing tenants to receive `PlanKey = starter` safely during migration. Full seed data for `Starter` and `Pro` is handled separately by the tenant-plan seeding work.

Startup seeding keeps the initial plans available and idempotently updates their defaults:

PLAN KEY	NAME	INTENT
starter	Starter	Default plan for new tenants with conservative limits.
pro	Pro	Larger customer plan with higher limits and self-service registration.

Initial feature keys:

FEATURE	STARTER	PRO
camp.enabled	true	true
branding.customLogo	false	true
registration.selfService	false	true

Initial settings:

SETTING	STARTER	PRO
limits.maxUsers	10	100
limits.maxCamps	3	25
registration.enabled	false	true
registration.requiresApproval	true	false
branding.theme	default	default

New tenants default to `PlanKey = starter` unless a later workflow explicitly allows a different plan to be supplied.

Effective Tenant Config

`ITenantConfigResolver` resolves the runtime configuration a tenant should receive. The resolver is registered in DI and can resolve by tenant id or tenant slug.

Effective config is built in this order:

SystemDefaults -> PlanDefaults -> TenantOverrides

Later layers override earlier layers.

The returned model contains:

FIELD	PURPOSE
<code>TenantId</code>	Catalog tenant identifier.
<code>TenantSlug</code>	Stable tenant slug.
<code>PlanKey</code>	Assigned plan key.
<code>PlanName</code>	Assigned plan name.
<code>Features</code>	Effective flat feature flag dictionary.
<code>Settings</code>	Effective structured JSON settings object.

Feature merge rules:

- feature dictionaries are flat;
- plan defaults override system defaults by key;
- tenant overrides override plan defaults by key;
- missing keys do not remove earlier defaults.

Settings merge rules:

- JSON objects are deep-merged;
- scalar values replace earlier scalar or object values;
- arrays replace earlier arrays and their contents are not merged;
- type conflicts are resolved by replacement from the later layer;
- empty dictionaries behave as empty layers;
- JSON `null` values are preserved as explicit override values when supplied by the stored JSON model.

Failure behavior:

CONDITION	RESULT ERROR CODE
Tenant does not exist or is soft-deleted	TENANT_NOT_FOUND
Assigned plan key does not resolve	TENANT_PLAN_NOT_FOUND
Assigned plan is inactive	TENANT_PLAN_INACTIVE

Tenant Config API

Tenant-facing clients can fetch effective config for the current tenant:

```
GET /api/v1/tenant/config
Authorization: Bearer <tenant-user-or-admin-token>
X-Tenant-Slug: acme
```

The route uses the tenant resolved from `X-Tenant-Slug`; the tenant id is not accepted in the URL for this endpoint.

Platform administrators can inspect a tenant directly:

```
GET /api/admin/tenants/{tenantId}/config
Authorization: Bearer <organizer-admin-token>
```

Example response:

```
{
  "tenantId": "11111111-1111-1111-1111-111111111111",
  "tenantSlug": "acme",
  "planKey": "pro",
  "planName": "Pro",
  "features": {
    "camps.enabled": true,
    "branding.customLogo": true,
    "registration.selfService": true
  },
  "settings": {
    "limits": {
      "maxUsers": 100,
      "maxCamps": 25
    },
    "registration": {
      "enabled": true,
      "requiresApproval": false
    },
    "branding": {
      "theme": "default"
    }
  }
}
```

Expected error behavior:

STATUS	CONDITION
401 Unauthorized	Missing or invalid bearer token.
403 Forbidden	Authenticated caller does not satisfy the endpoint policy or tenant access check.
400 Bad Request	Tenant-scoped route is missing X-Tenant-Slug .

STATUS

404 Not Found

CONDITION

Tenant, assigned plan, or active plan cannot be resolved.

Manual verification:

1. Seed `starter` and `pro` plans.
2. Create or onboard a tenant assigned to `pro`.
3. Add a tenant-level feature or setting override in the catalog.
4. Call `GET /api/v1/tenant/config` with `X-Tenant-Slug` set to that tenant.
5. Confirm the response contains system defaults, plan defaults, and tenant overrides with tenant overrides taking precedence.

Tenant Config Management

Platform administrators can manage plan defaults and tenant-level overrides.

Plan endpoints:

```
GET /api/admin/tenant-plans
GET /api/admin/tenant-plans/{planKey}
PUT /api/admin/tenant-plans/{planKey}
```

Tenant override endpoints:

```
GET /api/admin/tenants/{tenantId}/config-overrides
PUT /api/admin/tenants/{tenantId}/config-overrides
```

Plan keys are immutable. A plan update can change name, description, feature defaults, settings defaults, active state, and sort order.

Tenant override updates can change:

- assigned `planKey`;
- tenant-level feature overrides;

- tenant-level settings overrides.

Validation rules:

- feature keys must be non-empty;
- settings payloads must be JSON objects;
- assigned plan keys must resolve to active plans;
- invalid config payloads return `TENANT_CONFIG_INVALID`;
- missing tenants return `TENANT_NOT_FOUND`;
- missing or inactive plans return `TENANT_PLAN_NOT_FOUND` or `TENANT_PLAN_INACTIVE`.

Portal surfaces:

- `/admin/plans` lists and edits tenant plan defaults;
- `/admin/tenants` exposes each tenant's effective config and tenant-level override editor through the `Config` action.

Manual verification:

1. Open `/admin/plans` as an `organizer-admin`.
2. Edit a plan feature default or setting default and save.
3. Open `/admin/tenants`, choose a tenant assigned to that plan, and click `Config`.
4. Confirm the effective config reflects the plan default change.
5. Add a tenant feature or setting override and save.
6. Confirm the effective preview refreshes and tenant override values take precedence.

Tenant Statuses

STATUS	MEANING
Pending	Tenant exists in the catalog but provisioning is not complete.
Active	Tenant provisioning completed successfully.
Upgrading	Reserved for upgrade lifecycle state.

STATUS	MEANING
Failed	Provisioning failed or an unexpected onboarding error occurred.
Deleted	Tenant has been soft-deleted.

Tenant Resolution

`TenantResolutionMiddleware` resolves tenant context for tenant-scoped routes.

It skips platform endpoints such as:

- `/api/admin`;
- `/api/v1/tenants`;
- `/health`;
- `/scalar`;
- `/openapi`;
- root and common static paths.

For tenant-scoped requests, clients must send:

```
X-Tenant-Slug: acme
```

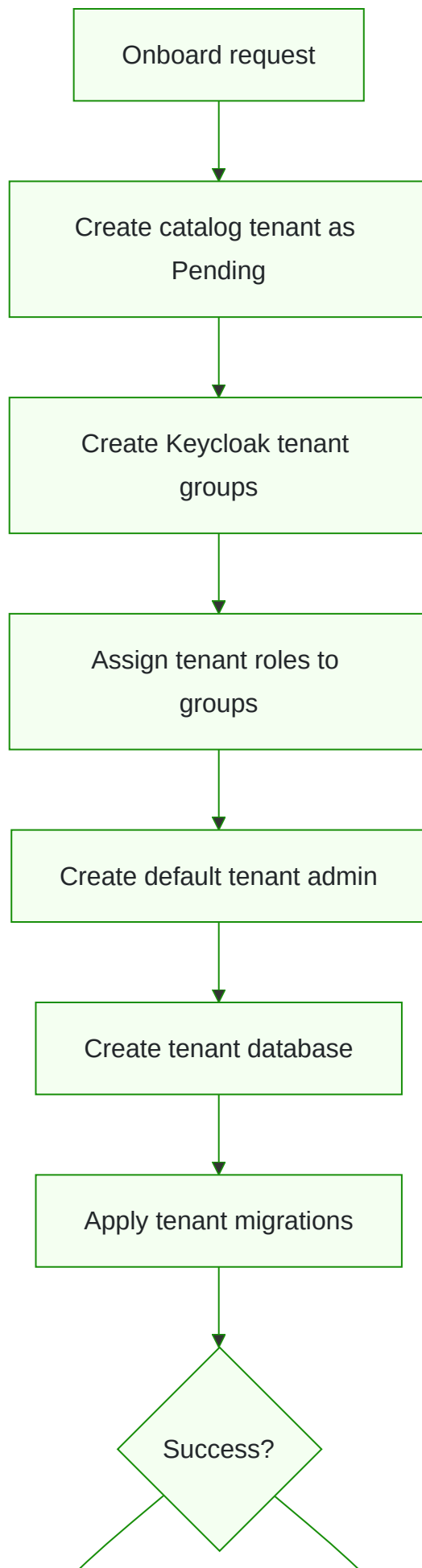
The middleware loads the tenant from the catalog and stores `TenantInfo` in `ITenantContext`. If the header is missing, the response is `400 Bad Request`. If the slug does not match a catalog tenant, the response is `404 Not Found`.

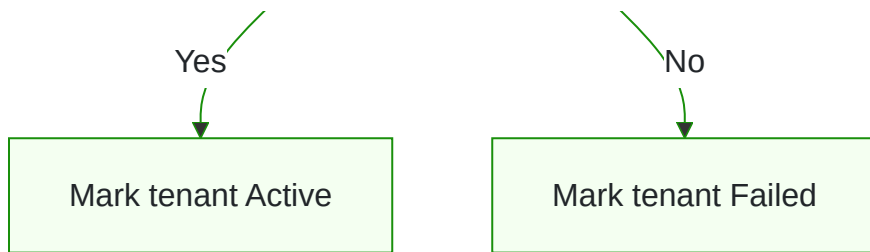
Tenant Database Resolution

`OrganizorDbContext` uses a fallback in-memory database when no tenant has been resolved. This supports startup, Scalar, migrations, and design-time tooling.

When a tenant is resolved, `PerTenantDbResolver` loads the catalog tenant, builds a PostgreSQL connection string for that tenant database, and `OrganizerDbContext` uses that database.

Tenant Onboarding Flow





not create Keycloak groups or a tenant database.

Hard deletion is intentionally guarded: the tenant must be soft-deleted before `TenantLifecycleService` runs the hard-delete workflow. That workflow removes the Keycloak tenant group and default admin user, drops the tenant database, and removes the catalog record.

Public API Behavior

The public API exposes tenant catalog endpoints and tenant-scoped config reads. Tenant catalog endpoints are platform/catalog routes, so tenant resolution does not require `X-Tenant-Slug` for them. They are still authenticated:

- `POST /api/v1/tenants` requires `organizor-admin`.
- `GET /api/v1/tenants` and `GET /api/v1/tenants/{slug}` require `organizor-admin`, `tenant-admin`, or `tenant-user`.

Tenant-scoped application endpoints, including `GET /api/v1/tenant/config`, must include the `X-Tenant-Slug` header unless the path is explicitly skipped by tenant resolution.

Internal tenant administration endpoints are documented separately in [Internal Operations](#).

Tenant Catalog

Create Tenant

```
POST /api/v1/tenants
Authorization: Bearer <organizor-admin-token>
Content-Type: application/json

{
  "name": "Acme",
  "slug": "acme"
}
```

Creates a catalog tenant record with `Pending` status and returns tenant metadata. This route does not provision Keycloak groups, a default tenant admin, or a tenant database. Use `POST /api/admin/tenants/onboard` for the full onboarding workflow.

Example response:

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "name": "Acme",
  "slug": "acme",
  "databaseName": "organizor_tenant_acme",
  "createdAt": "2026-06-09T10:30:00Z",
  "status": "Pending",
  "failureReason": null,
  "isDeleted": false
}
```

All tenant catalog responses use the same shape:

FIELD	MEANING
<code>id</code>	Tenant catalog identifier.
<code>name</code>	Human-readable tenant name.
<code>slug</code>	Stable slug used for lookup and tenant routing.
<code>databaseName</code>	Tenant database name assigned by the catalog.
<code>createdAt</code>	UTC catalog creation timestamp.
<code>status</code>	Current lifecycle status, such as <code>Pending</code> , <code>Active</code> , <code>Failed</code> , or <code>Deleted</code> .
<code>failureReason</code>	Provisioning or lifecycle failure detail when available.
<code>isDeleted</code>	Whether the catalog record has been soft-deleted. Public reads exclude deleted records.

Possible responses:

STATUS	MEANING
200 OK	Tenant catalog record was created.
401 Unauthorized	Missing or invalid bearer token.
403 Forbidden	Token does not satisfy the <code>organizer-admin</code> role policy.
409 Conflict	Duplicate tenant slug.
500 Internal Server Error	Unexpected server error. See Errors and Observability .

Get Tenant By Slug

```
GET /api/v1/tenants/acme
Authorization: Bearer <token>
```

Returns a tenant by slug using the tenant catalog response shape above, or `404 Not Found` when no tenant exists. Soft-deleted tenants are excluded.

List Tenants

```
GET /api/v1/tenants
Authorization: Bearer <token>
```

Returns all catalog tenants visible through the tenant repository using the tenant catalog response shape above. Soft-deleted tenants are excluded.

Tenant Config

```
GET /api/v1/tenant/config
Authorization: Bearer <tenant-token>
X-Tenant-Slug: acme
```

Returns the current tenant's effective config. Effective config is resolved from the tenant's assigned plan defaults plus tenant-specific overrides.

This endpoint requires `organizer-admin`, `tenant-admin`, or `tenant-user` through the `RequireTenantUserRole` policy. Tenant access middleware also verifies that the authenticated user belongs to the tenant identified by `X-Tenant-Slug`.

Platform Endpoints

ENDPOINT	PURPOSE
<code>/health</code>	Public health check endpoint.
<code>/openapi/v1.json</code>	OpenAPI document in development.
<code>/scalar</code>	Interactive OpenAPI/Scalar endpoint explorer in development.

OpenAPI And Scalar

When the API runs in development:

- OpenAPI JSON is available at `/openapi/v1.json`.
- Scalar is available at `/scalar`.

Use this article for behavior: routes, authorization expectations, tenant headers, response shapes, and operational meaning. Use the generated API reference only for C# type signatures.

Database Provisioning

Organizor uses one catalog database and one PostgreSQL database per tenant.

Catalog Database

`CatalogDbContext` stores tenant records in the `tenants` table and tenant plan definitions in the `tenant_plans` table.

Important constraints:

FIELD	CONSTRAINT
<code>Name</code>	Required, max length 200.
<code>Slug</code>	Required, max length 100, unique index.
<code>DatabaseName</code>	Required, max length 200.
<code>PlanKey</code>	Required, max length 100, defaults to <code>starter</code> , references <code>tenant_plans.key</code> .
<code>Features</code>	Required PostgreSQL <code>jsonb</code> , defaults to <code>{}</code> .
<code>Settings</code>	Required PostgreSQL <code>jsonb</code> , defaults to <code>{}</code> .
<code>IsDeleted</code>	Required, defaults to <code>false</code> .
<code>DeletionReason</code>	Optional, max length 500.

Important `tenant_plans` constraints:

FIELD	CONSTRAINT
<code>Key</code>	Required, max length 100, primary key.

FIELD	CONSTRAINT
Name	Required, max length 200.
Description	Optional, max length 500.
Features	Required PostgreSQL <code>jsonb</code> , defaults to <code>{}</code> .
Settings	Required PostgreSQL <code>jsonb</code> , defaults to <code>{}</code> .
IsActive	Required, defaults to <code>true</code> .
SortOrder	Required, defaults to <code>0</code> .
CreatedAt	Required UTC timestamp.

The tenant-config migration inserts a minimal `starter` plan row before adding the tenant plan foreign key. This keeps existing tenants migratable because their new `PlanKey` defaults to `starter`.

After migrations are applied, startup seeding idempotently creates or updates the initial tenant plans:

PLAN KEY	PURPOSE
<code>starter</code>	Default plan for new tenants with conservative limits.
<code>pro</code>	Larger customer plan with higher limits and more enabled features.

The seed data is managed in `SeedCatalogDbAsync`. It runs after `CatalogDbContext` migrations so it can enrich the minimal migration-created `starter` row and add `pro`.

Manual verification after startup:

```
select key, name, features, settings
from tenant_plans
where key in ('starter', 'pro')
order by sort_order;
```

Creating or onboarding a tenant without explicit plan selection should leave `tenants.plan_key` set to `starter`.

On startup, the API checks whether the catalog database exists, creates it when needed, and applies catalog migrations.

The full tenant onboarding workflow is documented in [Internal Operations](#). Database provisioning is one part of that workflow; Keycloak tenant access is provisioned before the tenant database is created.

Tenant Database Naming

Tenant databases use:

```
organizer_tenant_{slug}
```

Example:

```
organizer_tenant_acme
```

Slugs are normalized to lowercase before tenant records are written.

PostgreSQL Requirements

The configured PostgreSQL user must be able to:

- connect to the admin `postgres` database;

- create tenant databases;
- drop tenant databases during hard deletion;
- create extensions in tenant databases.

Tenant database creation currently uses:

```
CREATE DATABASE "{databaseName}" OWNER = postgres ENCODING = 'UTF8';
```

Each tenant database is initialized with:

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto";  
CREATE EXTENSION IF NOT EXISTS "citext";
```

Hard deletion uses:

```
DROP DATABASE IF EXISTS "{databaseName}" WITH (FORCE);
```

Migration Flow

`TenantMigrator` builds a tenant-specific connection string, checks pending migrations on `OrganizorDbContext`, applies them, and returns:

FIELD	MEANING
<code>Database</code>	Tenant database name.
<code>Success</code>	Whether migrations completed.
<code>AppliedMigrations</code>	Count of migrations pending before execution.
<code>Duration</code>	Migration runtime.

FIELD**MEANING**`Error`

Error message when migration fails.

Onboarding and Cleanup

During `POST /api/admin/tenants/onboard`, the database steps are:

1. create the tenant database named `organizer_tenant_{slug}`;
2. initialize required PostgreSQL extensions;
3. apply pending `OrganizerDbContext` migrations;
4. return success so the tenant can be marked `Active`.

If the migration result fails or onboarding throws unexpectedly, the catalog tenant is marked `Failed`.

During hard deletion, the tenant must already be soft-deleted. The hard-delete workflow drops the tenant database with `DROP DATABASE IF EXISTS ... WITH (FORCE)` after Keycloak cleanup and then removes the catalog record.

Keycloak Setup

This document describes the Keycloak configuration for the Organizer development environment. It includes realm setup, API/UI clients, tenant groups, claim mapping, user/role setup, and the admin-client permissions used by tenant onboarding.

The local Keycloak container is expected to run over HTTPS at

`https://localhost:8443`. See [Keycloak HTTPS Setup](#) before configuring clients.

Realm Setup

- Open Keycloak Admin Console.
- Realm dropdown -> **Create Realm**.
- Name: `organizer-dev`.
- Save.

API Client (`organizer-api`)

Settings:

- Client type: OpenID Connect
- Client ID: `organizer-api`
- Name: Organizer API

Required values:

- Client authentication: **ON**
- Authorization: OFF
- Standard flow: OFF
- Direct access grants: **ON**
- Implicit flow: OFF
- Service accounts roles: **ON**

Credentials:

- Open the **Credentials** tab.

- Copy the **Client Secret**.
- Store it in development tooling that requests tokens.

The API currently validates bearer tokens only; it does not read the client secret from application configuration. The client secret is only needed by tooling or services that request tokens with the `client_credentials` flow.

The infrastructure Keycloak admin client also uses this client secret during tenant onboarding. Configure these application settings from the same Keycloak client:

```
{
  "Keycloak": {
    "Authority": "https://localhost:8443/realms/organizer-dev",
    "BaseUrl": "https://localhost:8443",
    "Realm": "organizer-dev",
    "AdminClientId": "organizer-api",
    "AdminClientSecret": "<client-secret>"
  }
}
```

For tenant onboarding and hard deletion, the `organizer-api` service account must be allowed to use the Keycloak Admin REST API. Assign the required `realm-management` client roles for:

- reading groups and users;
- creating and deleting tenant groups;
- assigning realm roles to groups;
- creating the default tenant admin user;
- resetting the default tenant admin password;
- adding users to groups;
- deleting the default tenant admin user during hard deletion.

UI Client (`organizer-ui`)

Settings:

- Client type: OpenID Connect

- Client ID: `organizor-ui`
- Name: Organizor UI

Required values:

- Client authentication: OFF
- Authorization: OFF
- Standard flow: **ON**
- Direct access grants: OFF
- Implicit flow: OFF
- Service accounts roles: OFF

Development redirect URIs:

- `https://localhost:7025/*`
- `https://localhost:7025/authentication/login-callback`
- `http://localhost:5080/*`

Web origins:

- `https://localhost:7025`
- `http://localhost:5080`

Use `*` only for quick local testing. Explicit origins are preferable because they catch redirect and CORS mistakes earlier.

Tenant Groups

Tenant onboarding creates the group structure automatically when `POST /api/admin/tenants/onboard` is used. Manual setup is only needed for bootstrap or testing.

Group structure:

- `/tenants`
- `/tenants/acme`
- `/tenants/acme/users`
- `/tenants/acme/admins`

Each tenant has:

- `/users` group
- `/admins` group

On onboarding, Organizer assigns:

- `tenant-admin` to `/tenants/{slug}/admins`;
- `tenant-user` to `/tenants/{slug}/users`.

It also creates a default tenant admin user:

- Username: `{slug}-admin`
- Email: `{slug}-admin@organizer.local`
- Temporary password: `ChangeMe123!`
- Group: `/tenants/{slug}/admins`

During hard deletion, Organizer removes the tenant group `/tenants/{slug}` and deletes the default `{slug}-admin` user when it exists.

Tenant Membership Claim Mapper

Location:

- Clients -> `organizer-api`
- Client scopes -> `organizer-api-dedicated`
- Mappers -> Create

Values:

- Name: `tenant-membership`
- Mapper type: **Group Membership**
- Token Claim Name: `tenant`
- Full group path: **ON**
- Add to ID token: ON
- Add to access token: ON
- Add to userinfo: ON

User Setup

Create these realm roles before assigning users:

- `organizor-admin`
- `tenant-admin`
- `tenant-user`

Platform admin:

- Username: `admin@organizor.dev`
- Password: `Admin123!`
- Assign realm role: `organizor-admin`
- No tenant groups assigned

Tenant admin:

- Username: `admin@acme.dev`
- Password: `Admin123!`
- Assign realm role: `tenant-admin`
- Assign group: `/tenants/acme/admins`

Tenant user:

- Username: `user@acme.dev`
- Password: `Admin123!`
- Assign realm role: `tenant-user`
- Assign group: `/tenants/acme/users`

Token Testing

Token endpoint:

- `https://localhost:8443/realms/organizor-dev/protocol/openid-connect/token`

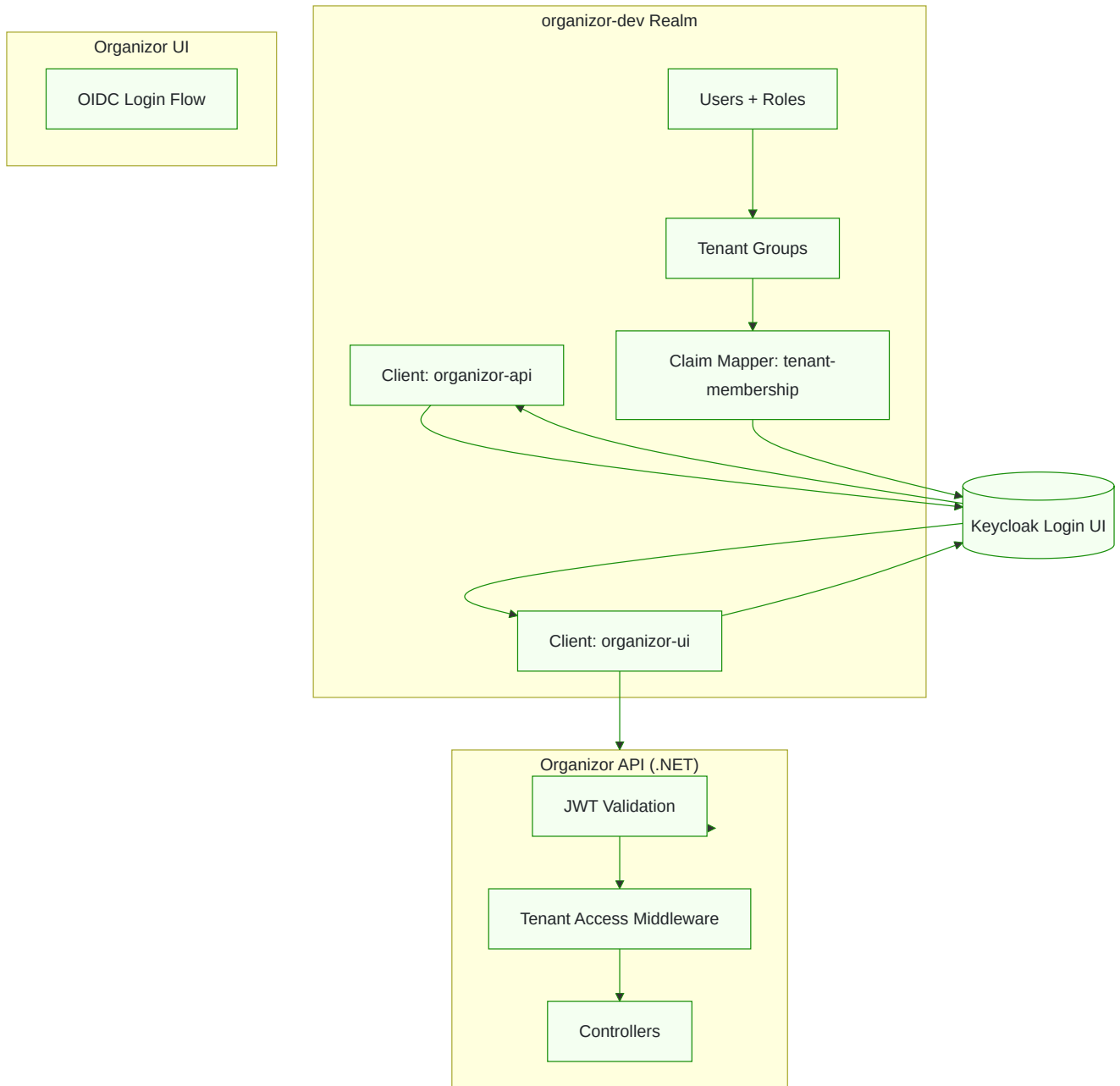
Expected role claim examples:

```
{
  "realm_access": {
    "roles": ["organizer-admin"]
  },
  "resource_access": {
    "organizer-api": {
      "roles": ["tenant-user"]
    }
  }
}
```

Example tenant claim:

```
"tenant": ["/tenants/acme/admins"]
```

Keycloak Architecture Diagram



Keycloak HTTPS Setup

The development Keycloak container runs over HTTPS at `https://localhost:8443`. The Blazor WebAssembly portal uses this endpoint for OIDC Authorization Code Flow with PKCE, and the API uses the same realm issuer when validating bearer tokens.

Why HTTPS Is Required

- The portal runs as a browser application and should not mix HTTPS portal/API calls with HTTP identity-provider redirects.
- OIDC redirect URIs must match the portal origin exactly.
- Keycloak 24 runs with explicit hostname and HTTPS configuration when started in production mode.
- Local HTTPS keeps browser, API, and identity-provider behavior closer to production.

Certificate

The compose file mounts a PKCS#12 certificate from:

```
keycloak/certs/keycloak.p12
```

Generate a trusted local certificate with `mkcert`:

```
mkcert -install  
mkcert -pkcs12 -p12-file .\keycloak\certs\keycloak.p12 localhost
```

`mkcert` PKCS#12 files use the password `changeit`, which matches the Docker Compose configuration.

Keep generated certificates out of source control. If the certificate is missing or not trusted, Keycloak may fail to start or the browser may reject the login page.

Docker Compose

The Keycloak service uses production mode with an HTTPS keystore:

```
keycloak:
  image: quay.io/keycloak/keycloak:24.0
  command:
    - start
    - --hostname=localhost
    - --https-port=8443
    - --https-key-store-file=/opt/keycloak/conf/keycloak.p12
    - --https-key-store-password=changeit
  volumes:
    - ./keycloak/certs/keycloak.p12:/opt/keycloak/conf/keycloak.p12
  ports:
    - "8443:8443"
```

Start or restart Keycloak from the repository root:

```
docker compose down
docker compose up -d
```

Open the admin console at:

```
https://localhost:8443
```

Development admin credentials are configured in `docker-compose.yml`:

```
admin / admin
```

Browser Trust

Chrome and Edge normally trust the `mkcert` root certificate through the Windows certificate store after `mkcert -install`.

If Edge still reports `ERR_CERT_AUTHORITY_INVALID`, import the mkcert root certificate manually into Trusted Root Certification Authorities:

```
%LOCALAPPDATA%\mkcert\rootCA.pem
```

Then restart the browser.

Expected Local Endpoints

COMPONENT	URL
Keycloak	<code>https://localhost:8443</code>
Keycloak realm issuer	<code>https://localhost:8443/realms/organizor-dev</code>
Portal UI	<code>https://localhost:7025</code>
API HTTPS profile	<code>https://localhost:7184</code>

The portal and API must both use the HTTPS Keycloak authority. Tokens issued by `https://localhost:8443/realms/organizor-dev` will not validate against an API configured for the old HTTP Keycloak endpoint. In local development, set both `Keycloak:Authority` and `Keycloak:BaseUrl` to HTTPS in API user secrets.

API User Secrets

For local development, verify the API user secrets use HTTPS:

```
dotnet user-secrets set "Keycloak:Authority"
"https://localhost:8443/realms/organizor-dev" --project
Code7Dev.Organizor.Api/Code7Dev.Organizor.Api.csproj
dotnet user-secrets set "Keycloak:BaseUrl" "https://localhost:8443" --
project Code7Dev.Organizor.Api/Code7Dev.Organizor.Api.csproj
```

Restart the API after changing these values. Running API processes keep the old configuration in memory.

The development API accepts the local Keycloak certificate for admin-client calls. Browser trust is still required for portal login.

Troubleshooting

SYMPTOM	LIKELY CAUSE	FIX
Keycloak does not start	Missing <code>keycloak/certs/keycloak.p12</code>	Generate the certificate with <code>mkcert</code>
Browser rejects Keycloak	<code>mkcert</code> root CA is not trusted	Run <code>mkcert -install</code> or import the manually.
Portal login redirects fail	Keycloak client redirect URI does not include <code>https://localhost:7025/*</code>	Update the <code>organizor-ui</code> client re URIs.
API returns 401 for	API <code>Keycloak:Authority</code> still points to HTTP	Set authority to <code>https://localhost:8443/realms/or dev</code> .

SYMPTOM	LIKELY CAUSE	FIX
portal tokens		

API Authentication

Overview

This document explains how the Organizer API authenticates requests using Keycloak-issued JWT tokens.

JWT Validation

The API validates tokens using the Keycloak realm:

- Authority: `https://localhost:8443/realms/organizer-dev`
- Audience validation: disabled for the current development setup
- Token type: Bearer

The authentication middleware:

- validates the token signature;
- validates the issuer;
- validates token expiration;
- populates `HttpContext.User`.

Audience validation is intentionally disabled in code while the Keycloak client and token shape are still being finalized. Once the API has a stable audience claim, enable audience validation and document the expected value here.

Authentication Flow

1. Frontend logs in via Keycloak.
2. Keycloak issues a JWT.
3. Frontend sends `Authorization: Bearer <token>`.
4. Frontend also sends `X-Tenant-Slug: acme` for tenant-scoped routes.
5. API validates the JWT.
6. `[Authorize]` attributes enforce authentication where applied.

Service Accounts

The `organizer-api` client can use the `client_credentials` flow for internal API-to-API communication. Service account tokens are identified by the `client_id` claim:

```
"client_id": "organizer-api"
```

Service accounts bypass tenant access checks.

WhoAmI Endpoint

Example response:

```
{
  "user": "admin@acme.dev",
  "tenant": null,
  "tenantClaims": [
    "/tenants/acme/admins"
  ],
  "roles": [
    "tenant-admin",
    "tenant-user"
  ],
  "claims": [
    { "type": "sub", "value": "..." },
    { "type": "tenant", "value": "/tenants/acme/admins" },
    { "type": "role", "value": "tenant-admin" }
  ]
}
```

Error Codes

- 401 Unauthorized -> Missing or invalid token.
- 403 Forbidden -> Token is valid, but the user is not allowed.
- 400 Bad Request -> Missing or invalid tenant header on tenant-scoped routes.

Test Request

```
GET https://localhost:7184/api/auth/whoami  
Authorization: Bearer <token>
```

Role-Based Authorization

Overview

The API authorizes requests with ASP.NET Core policies backed by Keycloak roles. Authentication validates the JWT first, tenant access middleware enforces tenant membership next, and endpoint authorization policies run after that.

Roles

ROLE	PURPOSE
<code>organizer-admin</code>	Platform administrator. Can manage tenants and call admin operations.
<code>tenant-admin</code>	Tenant administrator. Can use tenant-aware application endpoints and tenant catalog reads.
<code>tenant-user</code>	Tenant member. Can use shared tenant-aware application endpoints and tenant catalog reads.

JWT Role Sources

The API maps these Keycloak claim shapes to ASP.NET Core role claims:

```
{
  "realm_access": {
    "roles": ["organizer-admin"]
  },
  "resource_access": {
    "organizer-api": {
      "roles": ["tenant-user"]
    }
  }
}
```

```
}  
}
```

Only client roles under `resource_access.organizor-api.roles` are promoted. Roles for other clients, such as `account`, are ignored for API authorization.

Policies

POLICY	ACCEPTED ROLES
<code>RequireOrganizorAdminRole</code>	<code>organizor-admin</code>
<code>RequireTenantUserRole</code>	<code>organizor-admin</code> , <code>tenant-admin</code> , <code>tenant-user</code>
<code>RequireTenantAdminRole</code>	<code>organizor-admin</code> , <code>tenant-admin</code>

Use policies for new endpoints unless the endpoint is intentionally authentication-only.

```
[Authorize(Policy = AuthorizationPolicies.RequireOrganizorAdminRole)]
```

Current Endpoint Enforcement

ENDPOINT AREA	POLICY
<code>/api/admin/tenants/*</code>	<code>RequireOrganizorAdminRole</code>
<code>/api/admin/tenant-plans/*</code>	<code>RequireOrganizorAdminRole</code>
<code>POST /api/v1/tenants</code>	<code>RequireOrganizorAdminRole</code>
<code>GET /api/v1/tenants</code>	<code>RequireTenantUserRole</code>

ENDPOINT AREA	POLICY
GET /api/v1/tenants/{slug}	RequireTenantUserRole
GET /api/v1/tenant/config	RequireTenantUserRole
GET /api/auth/whoami	authenticated user only

Tenant Interaction

Role checks do not replace tenant checks. For tenant-scoped routes, a user must pass both:

- Tenant authorization in `TenantAccessMiddleware`.
- Endpoint role or policy authorization in ASP.NET Core.

This means a `tenant-user` token for `acme` is still denied when it sends `X-Tenant-Slug: contoso`.

Assigning Roles in Keycloak

Realm roles are assigned from the user details screen:

1. Open Keycloak Admin Console.
2. Select the `organizer-dev` realm.
3. Open Users and select the user.
4. Open Role mapping.
5. Assign one of `organizer-admin`, `tenant-admin`, or `tenant-user`.

Client roles can also be assigned under the `organizer-api` client. If client roles are used, verify they appear in `resource_access.organizer-api.roles`.

Test Matrix

SCENARIO	EXPECTED
organizer-admin calls /api/admin/tenants	Allowed
tenant-user calls /api/admin/tenants	403 Forbidden
tenant-admin calls GET /api/v1/tenants	Allowed
tenant-user calls GET /api/v1/tenants	Allowed
tenant-user calls GET /api/v1/tenant/config for their tenant	Allowed
tenant-user calls /api/admin/tenant-plans	403 Forbidden
Token has no Organizer role and calls protected endpoint	403 Forbidden
Valid role with wrong tenant slug on tenant-scoped route	403 Forbidden

Tenant Authorization

Overview

Tenant-scoped API routes require both a valid JWT and an `X-Tenant-Slug` header. The API resolves the header to a catalog tenant, validates that the JWT is allowed to use that tenant, then lets controller authorization policies run.

Tenant Resolution

Tenant-scoped routes must include:

```
X-Tenant-Slug: acme
```

The tenant resolution middleware skips platform and infrastructure routes, including:

- `/api/admin`
- `/api/v1/tenants`
- `/api/auth`
- `/health`
- `/openapi`
- `/scalar`
- `/favicon`
- `/robots`

Resolution steps:

1. Middleware reads `X-Tenant-Slug`.
2. Middleware looks up the tenant in the catalog database.
3. Middleware sets `tenantContext.Current`.
4. Middleware logs tenant metadata.

JWT Tenant Claim

Tenant membership is represented by one or more `tenant` claims. The middleware accepts either direct slugs or Keycloak-style paths:

```
{  
  "tenant": ["/tenants/acme/admins", "/tenants/contoso/users"]  
}
```

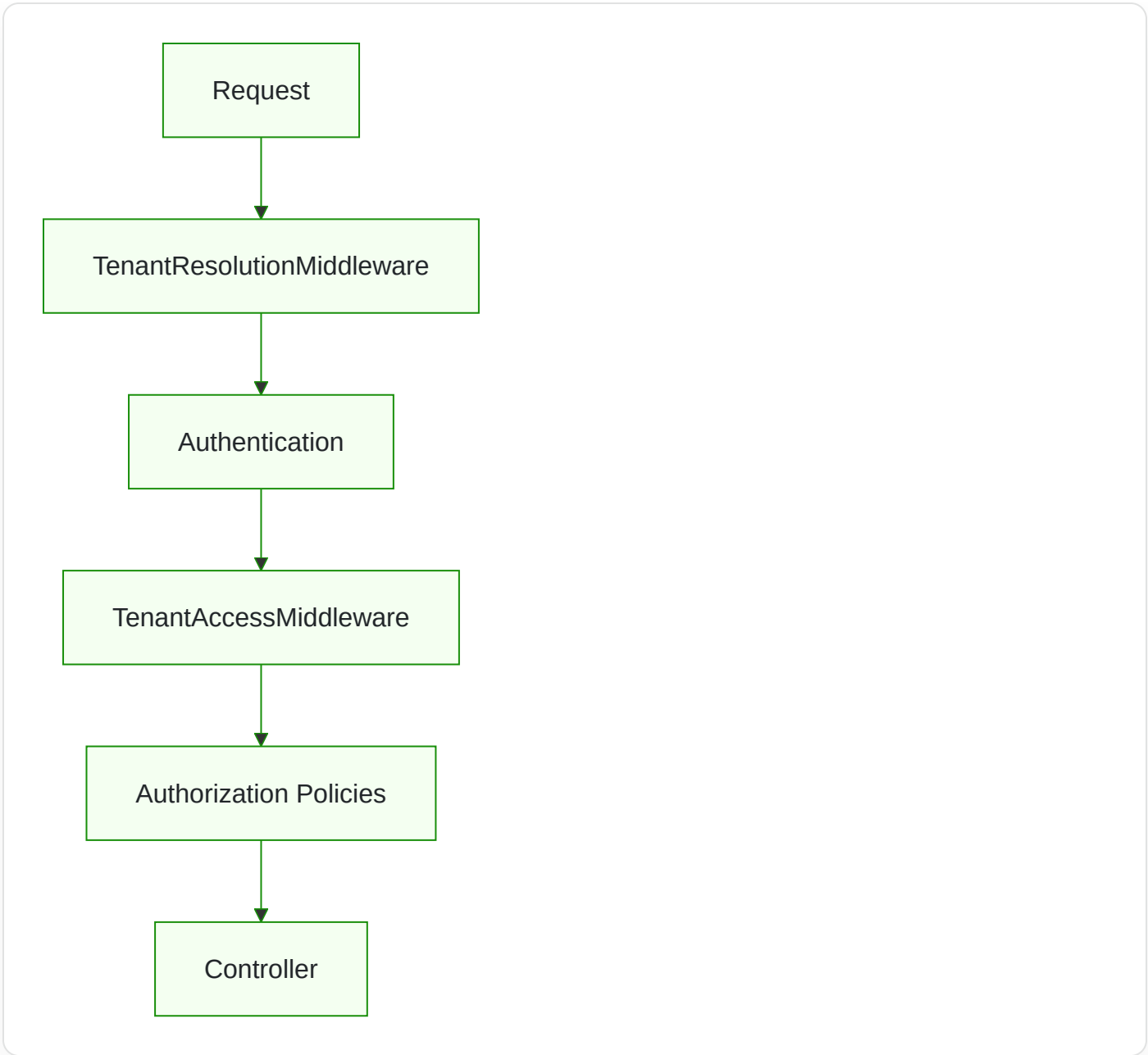
The slug is extracted from the segment after `/tenants/`. For `/tenants/acme/admins`, the effective tenant slug is `acme`.

Enforcement Rules

For tenant-scoped routes:

- Missing or invalid `Authorization` returns `401 Unauthorized` when the endpoint requires `[Authorize]`.
- Missing `X-Tenant-Slug` returns `400 Bad Request`.
- Unknown `X-Tenant-Slug` returns `404 Not Found`.
- A JWT with tenant claims must include the same slug as `X-Tenant-Slug`.
- A JWT tenant/header mismatch returns `403 Forbidden`.
- A JWT without tenant claims is treated as a platform superadmin token and bypasses tenant membership checks.
- The `organizer-api` service account, identified by `client_id = organizer-api`, bypasses tenant membership checks.
- Matching tenant JWTs are still checked through `ITenantAccessService.UserHasAccessAsync(userId, tenantId)`.
- If `ITenantAccessService` denies access, the API returns `403 Forbidden`.

Middleware Chain



HTTP Test Matrix

SCENARIO	AUTHORIZATION	X-TENANT-SLUG	EXPECTED
Tenant token for <code>acme</code> calls <code>acme</code>	tenant JWT	<code>acme</code>	Allowed if <code>ITenantAccessService</code> allows

SCENARIO	AUTHORIZATION	X-TENANT-SLUG	EXPECTED
Tenant token for <code>acme</code> calls <code>contoso</code>	tenant JWT	<code>contoso</code>	403 Forbidden
Superadmin token has no tenant claim	superadmin JWT	any valid slug	Allowed
Service account token	<code>client_id = organizer-api</code>	any valid slug	Allowed
Missing tenant header on tenant route	valid JWT	omitted	400 Bad Request
Unknown tenant header on tenant route	valid JWT	unknown slug	404 Not Found
Missing token on authorized route	omitted	valid slug	401 Unauthorized
<code>ITenantAccessService</code> denies matching user	tenant JWT	matching slug	403 Forbidden

Sample Requests

Tenant-scoped request:

```
GET http://localhost:5100/api/some-tenant-scoped-route
Authorization: Bearer <tenant-token>
X-Tenant-Slug: acme
Accept: application/json
```

Cross-tenant denial:

```
GET http://localhost:5100/api/some-tenant-scoped-route
Authorization: Bearer <token-with-tenant-acme>
X-Tenant-Slug: contoso
Accept: application/json
```

Superadmin bypass:

```
GET http://localhost:5100/api/some-tenant-scoped-route
Authorization: Bearer <token-without-tenant-claim>
X-Tenant-Slug: acme
Accept: application/json
```

Configuration

Organizor reads standard ASP.NET Core configuration from `appsettings.json`, environment-specific files, user secrets, and environment variables.

Catalog Connection String

`CatalogDbContext` expects a connection string named `Catalog`.

Example shape:

```
{
  "ConnectionStrings": {
    "Catalog":
      "Host=localhost;Port=5432;Database=organizor_catalog;Username=postgres;Password=postgres"
  }
}
```

Tenant PostgreSQL Settings

`ConnectionStringBuilder` builds tenant and admin connection strings from the `Postgres` section.

```
{
  "Postgres": {
    "Host": "localhost",
    "Port": "5432",
    "User": "postgres",
    "Password": "postgres"
  }
}
```

The admin connection uses the `postgres` database. Tenant connections use the tenant database name stored in the catalog.

Keycloak

The API requires Keycloak settings for two separate concerns:

- JWT validation uses `Keycloak:Authority`.
- Tenant onboarding uses the Keycloak admin client settings to create tenant groups, assign roles, and create the default tenant admin user.

Example development shape:

```
{
  "Keycloak": {
    "Authority": "https://localhost:8443/realms/organizer-dev",
    "BaseUrl": "https://localhost:8443",
    "Realm": "organizer-dev",
    "AdminClientId": "organizer-admin",
    "AdminClientSecret": "<admin-client-secret>"
  }
}
```

`Authority` must match the token issuer exactly. The local Keycloak container runs at `https://localhost:8443`, so tokens from the Blazor portal have issuer `https://localhost:8443/realms/organizer-dev`.

A mismatch between `http` and `https` is enough to make API authentication fail with `401 Unauthorized`.

The development API configures the Keycloak admin `HttpClient` to accept the local self-signed Keycloak certificate. This is development-only behavior and should not be used for production identity-provider calls.

Store client secrets in user secrets, environment variables, or deployment secrets rather than committing them to source.

Portal UI

The Blazor WebAssembly portal uses Keycloak for OIDC login and calls the API with bearer tokens. Admin tenant management uses Radzen components and `AdminTenantService` over the named `api` `HttpClient`.

Current local development endpoints:

SETTING	VALUE
Portal URL	<code>https://localhost:7025</code>
Keycloak authority	<code>https://localhost:8443/realms/organizer-dev</code>
Keycloak client ID	<code>organizer-ui</code>
Login callback	<code>https://localhost:7025/authentication/login-callback</code>
API base URL	<code>https://localhost:7184</code>

These values must stay aligned with the `organizer-ui` client redirect URIs and web origins in Keycloak.

Quartz

Quartz uses the `Catalog` connection string for its persistent store. The same catalog database stores tenant records and Quartz scheduler tables.

The API registration configures:

- PostgreSQL-backed Quartz persistence;
- Newtonsoft JSON serialization for persisted job data;
- clustering support;
- hosted service startup for Quartz background processing.

Tenant job scheduling is not enabled on application startup yet. `TenantJobScheduler` is available for future real tenant jobs, but the startup scheduling block is currently

commented out until those jobs exist.

Logging

Serilog is configured in `appsettings.json`.

Current sinks:

- console;
- rolling file logs under `logs/log-.txt`.

Log output includes tenant and request correlation properties when available:

- `Id`;
- `Slug`;
- `DatabaseName`;
- `CorrelationId`.

CORS

The API selects a CORS policy by environment:

- `DevCors` in development allows any origin, header, and method.
- `ProdCors` outside development must be configured with explicit allowed origins before browser clients can call the API.

The current production policy has no origins configured. Treat that as a deployment checklist item, not a complete production setting.

OpenAPI and Scalar

OpenAPI and Scalar endpoints are mapped only in development. This keeps the production surface smaller while retaining local API exploration during development.

Localization

Organizor provides multilingual support for error responses using a simple, predictable, API-friendly localization pipeline.

Supported Languages

Organizor currently supports:

- `en` - English (fallback)
- `nl` - Dutch

Additional languages can be added by creating new resource files in the `Code7Dev.Organizor.Api/Resources` folder.

How Localization Works

Localization is driven by the standard `Accept-Language` HTTP header.

Example:

```
Accept-Language: nl
```

The API resolves the culture for the request and uses it to translate error messages. If a translation is missing, the system falls back to English (`en`).

Key Principles

- Application services return error codes only, such as `TENANT_NOT_FOUND`.
- The API middleware maps those codes to localized resource strings.
- No hardcoded user-facing messages exist in the domain or application layers.
- Localization is deterministic and client-controlled.

Culture Resolution

Organizor uses the following resolution order:

- `Accept-Language` header.
- Fallback to English (`en`).

This keeps behavior predictable and client-controlled.

Error Code Translation

Each error code corresponds to a resource key.

Examples:

- `TENANT_NOT_FOUND`
- `TENANT_ALREADY_EXISTS`
- `UNEXPECTED_ERROR`

These keys exist in:

- `Code7Dev.Organizor.Api/Resources/Shared.en.resx`
- `Code7Dev.Organizor.Api/Resources/Shared.nl.resx`

The middleware retrieves the localized string and injects it into the RFC 7807 problem response.

Example Request and Response

Request:

```
GET /api/v1/tenants/unknown
Authorization: Bearer <token>
Accept-Language: nl
```

Response:

```
{  
  "type": "https://httpstatuses.com/404",  
  "title": "Tenant niet gevonden",  
  "status": 404,  
  "error": "TENANT_NOT_FOUND",  
  "correlationId": "abc123"  
}
```

The error code remains stable and language-independent. The title is localized based on the request culture.

Adding New Translations

To add a new language:

1. Create a new `.resx` file, for example `Shared.fr.resx`.
2. Add the same error keys as in the English file.
3. Provide translated values.
4. Register the culture in `AddApiLocalization`.
5. Restart the API.

Errors and Observability

Correlation IDs

Every request passes through `CorrelationIdMiddleware`.

Clients can provide a correlation ID:

```
X-Correlation-ID: request-123
```

If the header is missing, the API generates a GUID. The final value is returned in the response header:

```
X-Correlation-ID: 2ad71e7a-13cb-4ef8-a4c7-f3a37ad9dc71
```

The same value is pushed into Serilog context as `CorrelationId`.

Error Shape

Unhandled exceptions are returned as RFC 7807-style problem details:

```
{
  "type": "https://httpstatuses.com/500",
  "title": "An unexpected error occurred.",
  "status": 500,
  "detail": "Exception message",
  "instance": "/api/v1/tenants",
  "correlationId": "2ad71e7a-13cb-4ef8-a4c7-f3a37ad9dc71"
}
```

The response content type is:

```
application/problem+json
```

NOTE

The current middleware includes exception messages in `detail`. Treat that as useful during development, but review it before exposing production environments.

Tenant Resolution Errors

Tenant-scoped requests require:

```
X-Tenant-Slug: acme
```

Missing tenant slug returns `400 Bad Request` with a plain text response. Unknown tenant slug returns `404 Not Found` with a plain text response.

Log Context

Serilog output is configured to include:

PROPERTY

```
CorrelationId
```

```
Id
```

```
Slug
```

```
DatabaseName
```

SOURCE

Request header or generated GUID.

Resolved tenant ID.

Resolved tenant slug.

Tenant database selection when available.

PROPERTY	SOURCE
SourceContext	Logger source in file logs.

Internal Operations

This page documents internal routes and workflows for developers and operators.

Onboard Tenant

```
POST /api/admin/tenants/onboard
Authorization: Bearer <organizor-admin-token>
Content-Type: application/json

{
  "name": "Acme",
  "slug": "acme"
}
```

The onboarding flow:

1. normalizes the slug to lowercase;
2. creates a catalog tenant with `Pending` status;
3. provisions Keycloak tenant groups under `/tenants/{slug}`;
4. assigns `tenant-admin` to `/tenants/{slug}/admins` and `tenant-user` to `/tenants/{slug}/users`;
5. creates a default tenant admin user named `{slug}-admin` with a temporary password of `ChangeMe123!`;
6. creates the tenant database using `organizor_tenant_{slug}`;
7. initializes PostgreSQL extensions;
8. applies tenant migrations;
9. marks the tenant `Active` on success or `Failed` on provisioning failure.

Example response:

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "name": "Acme",
```

```
"slug": "acme",
"databaseName": "organizor_tenant_acme",
"createdAt": "2026-06-09T10:30:00Z",
"status": "Active",
"failureReason": null,
"isDeleted": false
}
```

Query Tenants For Administration

```
GET /api/admin/tenants
Authorization: Bearer <organizor-admin-token>
```

Returns all tenant catalog records, including soft-deleted tenants. This differs from `GET /api/v1/tenants`, which returns only non-deleted records.

```
GET /api/admin/tenants/{id}
Authorization: Bearer <organizor-admin-token>
```

Returns one tenant by catalog ID, including soft-deleted records. These query endpoints are used by the portal tenant-management surface and are protected by the same `RequireOrganizorAdminRole` policy as lifecycle operations.

Example response:

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "name": "Acme",
  "slug": "acme",
  "databaseName": "organizor_tenant_acme",
  "createdAt": "2026-06-09T10:30:00Z",
  "status": "Active",
  "failureReason": null,
}
```

```
"isDeleted": false
}
```

Tenant Config Administration

```
GET /api/admin/tenants/{id}/config
Authorization: Bearer <organizor-admin-token>
```

Returns the effective config for one tenant. Effective config is resolved from the tenant's assigned plan defaults plus any tenant-level overrides.

```
GET /api/admin/tenants/{id}/config-overrides
Authorization: Bearer <organizor-admin-token>
```

Returns only explicit tenant-level overrides.

```
PUT /api/admin/tenants/{id}/config-overrides
Authorization: Bearer <organizor-admin-token>
Content-Type: application/json

{
  "maxUsers": 25,
  "maxProjects": null,
  "allowAdvancedReporting": true
}
```

`null` clears an override and lets the tenant inherit the value from its plan. Override updates validate the resulting config before saving.

Tenant users can view their own effective config through the tenant-scoped endpoint:

```
GET /api/v1/tenant/config
Authorization: Bearer <tenant-token>
X-Tenant-Slug: acme
```

This endpoint requires tenant access and the `RequireTenantUserRole` policy.

Tenant Plan Administration

```
GET /api/admin/tenant-plans
Authorization: Bearer <organizer-admin-token>
```

Returns available tenant plans and their default feature settings.

```
PUT /api/admin/tenant-plans/{planKey}
Authorization: Bearer <organizer-admin-token>
Content-Type: application/json

{
  "maxUsers": 100,
  "maxProjects": 50,
  "allowAdvancedReporting": true
}
```

Plan updates validate the resulting default settings before saving. Existing tenant overrides remain intact, so tenants can continue to diverge from their plan where an override is set.

Upgrade Tenant

```
POST /api/admin/tenants/{id}/upgrade
```

```
Authorization: Bearer <organizer-admin-token>
```

Runs pending migrations for the tenant database.

Example response:

```
{
  "database": "organizer_tenant_acme",
  "success": true,
  "appliedMigrations": 2,
  "duration": "00:00:01.2340000",
  "error": null
}
```

Soft Delete Tenant

```
DELETE /api/admin/tenants/{id}
Authorization: Bearer <organizer-admin-token>
Content-Type: application/json

{
  "reason": "Customer cancellation"
}
```

Soft deletion marks the tenant as deleted, stores deletion metadata, and sets status to `Deleted`. The tenant database is not dropped.

Hard Delete Tenant

```
POST /api/admin/tenants/{id}/hard-delete
Authorization: Bearer <organizer-admin-token>
```

Hard deletion requires the tenant to already be soft-deleted. It:

1. deletes the default Keycloak admin user named `{slug}-admin` when it exists;
2. deletes the Keycloak tenant group `/tenants/{slug}` when it exists;
3. drops the tenant database;
4. removes the catalog record.

Keycloak cleanup tolerates missing groups or users so a partially-provisioned tenant can still be cleaned up.

Portal Admin Tenant Control

The Blazor portal exposes the tenant operations above through `/admin/tenants` for users with the `organizer-admin` role.

The page uses `AdminTenantService`, which calls the same admin endpoints documented in this article. Operators can:

- view all tenants, including soft-deleted tenants;
- filter the catalog by name, slug, database, or status;
- onboard a new tenant;
- inspect effective tenant config;
- edit tenant config overrides;
- manage plan default settings from `/admin/plans`;
- run a tenant upgrade;
- soft-delete a tenant with a reason;
- hard-delete a tenant after it has been soft-deleted.

The portal does not bypass API authorization. Each operation still requires a bearer token accepted by `RequireOrganizerAdminRole`.

Scheduled Tenant Jobs

Organizer is wired for Quartz recurring background work, but tenant job scheduling is not currently enabled on startup. `Program.cs` contains a commented-out scheduling block for future use.

The scheduler:

1. loads all tenants from the catalog database;
2. creates a durable `SyncTenantJob` with job key `sync-{tenantId}` in the `tenant-jobs` group;
3. stores `TenantId` and `TenantSlug` in the Quartz job data map;
4. creates a trigger named `sync-{tenantId}-trigger` in the `tenant-triggers` group;
5. schedules the job when it is new, or replaces and reschedules it when it already exists.

Current trigger cadence:

```
0/10 * * * * ?
```

That Quartz cron expression would run the tenant synchronization job every 10 seconds when scheduling is enabled.

`SyncTenantJob` reads the tenant data from the job data map, sets `ITenantContext`, and then runs tenant-scoped work through `OrganizorDbContext`. The current implementation logs execution and contains placeholder synchronization work.

Quartz persistence is stored in the catalog database and uses the PostgreSQL tables added by the `AddQuartzTables` migration. The API configures Quartz with the `Catalog` connection string, JSON serialization, and clustering enabled.

Documentation Deployment

The repository contains a GitHub Actions workflow at `.github/workflows/docfx-deploy.yml`.

On pushes to `master`, it:

1. checks out this repository;
2. installs DocFX;
3. builds `docs/docfx.json`;

4. checks out the public documentation repository;
5. copies `docs/_site` into that repository;
6. commits and pushes the generated documentation.

The workflow uses the `DOCS_DEPLOY_KEY` secret to push to the public docs repository.

Sitemap and Crawlers

DocFX generates `docs/_site/sitemap.xml` from the `build.sitemap.baseUrl` value in `docs/docfx.json`.

The documentation source includes `robots.txt`, which is copied into the generated site and points crawlers to the sitemap.